技术讲解与检查指南

这份指南用于亲自理解和检查本项目。先看演示结果，再填一份问卷，最后把网页上的行为对应到后端代码和数据库。所有操作都在本机完成。

一 先把系统想成三部分

部分	在项目中的位置	它负责什么
前端	templates/ 和 static/	显示页面、收集输入、调用接口、展示错误与结果。
后端	app.py、logic.py	判断是谁在操作、是否有权限、答案是否合法、下一题是哪题，并决定保存什么。
数据库	db.py 连接 MongoDB，结构见 schema.json	长期保存账号、问卷定义和提交的答卷。

例如，你点击“下一题”：fill.js 读取输入 → app.py 接收请求 → logic.py 校验并计算下一题 → 后端返回结果 → fill.js 显示对应题目。这个动作还没有保存答卷。

你点击“提交问卷”：后端从第一题重新走一遍路径、检查答案，然后通过 PyMongo 向 MongoDB 插入一份答卷。统计页再读取这些答卷计算结果。

二 运行与演示账号

如果当前已经能打开 http://127.0.0.1:5000/，直接使用即可。服务停止后按根目录 README，先启动 MongoDB，再运行 app.py；不要在同一端口重复启动数据库。

演示数据可以用以下命令创建，重复执行不会覆盖已有数据：

```
.\.venv\Scripts\python.exe scripts\seed_demo.py
```

账号	密码	用途
demo_owner	Demo12345!	创建者，查看“演示问卷：校园生活调查”的统计。
demo_user	Demo12345!	填写者，访问分享链接。自己的问卷列表为空是正常的，因为它没有创建问卷。

脚本会打印该问卷的填写链接和统计链接。也可以登录 demo_owner，从列表复制填写链接。演示问卷是匿名、允许重复填写的，后续提交会使统计变化。

三 先核对两份已知答卷

先登录 demo_owner，查看演示问卷的统计。初次生成数据时应看到：

题目	初始结果	为什么
是否填写兴趣题	填写 1 次，跳过 1 次	两份答案分别选了 yes、no。
水果	回答 1 人次，苹果和香蕉各 1 次，橙子 0 次	另一份答卷跳过了水果题。
年龄	20 和 16，平均值 18	(20 + 16) / 2 = 18。
课程学习建议	只有“增加上机练习”	16 岁的答卷跳过这道必答题。
其他意见	只有“页面清楚”	另一份选填留空，不算有效填写。

明细应显示“匿名”。切换“统计范围”为年龄，页面仅显示这一题的统计及对应明细。

另有一份由助手通过浏览器逐步创建的“浏览器检查问卷”，属于 browser_check 测试账号，年龄为 20、40，平均值为 30。测试报告和截图引用的是它；请与上面的演示问卷区分。

四 亲自走三种跳转路径

退出创建者，登录 demo_user，打开演示问卷的填写链接。可以分别提交以下三次，理解规则如何改变路径。

1. **单选跳转**：第一题选“跳过兴趣题” → 直接进入年龄，水果题不出现。年龄填 16 → 又直接进入“其他意见”，课程建议题不出现。
2. **多选跳转**：第一题选“填写兴趣题” → 水果勾选“橙子” → 直接进入“其他意见”，年龄和课程建议都不出现。
3. **顺序与数字判断**：第一题选“填写兴趣题” → 水果只选“苹果” → 年龄填 20 → 正常进入课程学习建议，至少填两个字符 → 进入其他意见并提交。

第三条中，年龄不小于 18，没有命中数字跳转条件，因此按顺序进入下一题。

过程中顺便检查：不回答必答题、勾选三种水果、年龄填 121 或 1.5、建议只填一个字，都应出现明确错误。修正后可以继续。使用“上一题”改选时，之后的路径会重新确定。

这些操作会新增答卷，因此不要再要求统计保持初始的两份。刷新统计后，按自己刚提交的答案核算变化。

五 自己创建一份问卷

注册你自己的账号，新建一个问卷，设置标题和说明。添加：

1. 单选“是否填写详细说明”，两个选项“填写”“跳过”。
2. 文本“详细说明”，设为必答，最少 2 个字符。
3. 数字“评分”，设为 0-10 的整数。

在第一题添加规则：“跳过” → 跳到第 3 题。保存草稿，离开后再打开，确认题目和规则仍存在。最后发布并复制链接，用另一个账号填写。

这时你实际完成了“前端生成问卷配置 → 后端检查配置 → MongoDB 保存定义”。发布后不能改题；列表仍能调整截止时间和关闭问卷。关闭操作放在最后检查，因为关闭后不再收集答卷。

六 在 MongoDB 中找到同一份数据

在终端连接本地数据库：

```
mongosh "mongodb://127.0.0.1:27017/survey_stage1"
```

以下均是读取命令，不会修改数据：

```
show collections
db.users.find({}, {username: 1, created_at: 1})
var s = db.surveys.findOne({demo_tag: "stage1_walkthrough_v1"})
s.title
s.questions
db.responses.find({survey_id: s._id}, {answers: 1, submitted_at: 1})
db.responses.getIndexes()
```

你会发现：

- 用户有独立 _id。问卷的 owner_id 对应创建者的 _id。
- questions 是问卷文档内的数组，题目、选项、校验和跳转都在里面。
- 每次提交是一份新的 responses 文档，survey_id 指向问卷。
- 答案键是 q_age 等稳定标识。数值年龄保存为数字，而非显示文字。

- 16 岁那份答卷没有 q_fruits 和 q_advice；q_feedback 是 null。这分别表示跳过和选填留空。

可以直接在数据库核对年龄平均值：

```
db.responses.aggregate([
  {$match: {survey_id: s._id, "answers.q_age": {$type: "number"}}},
  {$group: {_id: null, answered: {$sum: 1}, average: {$avg: "$answers.q_age"}}}
])
```

初始数据返回回答数 2、平均值 18。新增答卷后按实际数据变化。网页统计由后端读取答卷后计算，这个数据库查询是独立的核对方法。

七 看一次前后端请求

如果使用 Chrome 或 Edge，可按 F12，打开 Network，再填写一题并点击“下一题”。找到以 check-answer 结尾的请求：

```
{"question_id":"q_age","value":"16"}
```

后端应返回：

```
{"answer":16,"next_id":"q_feedback"}
```

这里能看到两个变化：输入框给的是字符串，后端把它规范化成数字；下一题由后端根据数据库中的规则决定。此时 responses 数量还没有增加，最后提交成功后才增加。

八 按这个顺序看代码

1. 看 static/fill.js 的提交事件，理解输入如何通过 fetch 发给后端。
2. 看 app.py 的 check_answer 和 submit_response，理解登录、问卷状态及保存的分工。
3. 看 logic.py 的 validate_answer，理解不同题型如何检查。
4. 看 next_question_id，理解规则按顺序匹配；再看 validate_submission，手动跟随 q_path → q_age → q_feedback 走一遍循环。
5. 看 db.py 的 init_db，对应 MongoDB 中的三个集合与索引；看 schema.json，理解数据库层还能阻止字段格式错误。
6. 看 build_statistics，用两份已知答案核对计数和平均值。

无需一开始读完整个项目。先沿着一次提交的调用顺序，通常更容易理解。

九 运行自动化测试

MongoDB 保持运行，在项目根目录执行：

```
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

23 项测试检查纯逻辑，26 项检查真实 MongoDB 与 HTTP 接口。集成测试使用随机独立数据库，并在结束时清理它；不会清理演示问卷。具体输入、预期、实际结果见 测试报告.md。

测试通过说明这些具体场景得到验证。你可以在报告的 AI 使用部分补记自己实际进行的检查、理解和修改；不要把尚未亲手做过的操作写成已完成。